A beginner's guide to C and Uzebox games: Pong in video mode 748

This tutorial teaches you enough C to write a complete game for the Uzebox: a version of the classic bat-and-ball game Pong, using video mode 748. No previous programming experience is assumed. We build the game in four steps, and every step is a program you can build and run.

Each step's complete source, with its `Makefile` and `m74cfg.h`, is in this repository's `step1`, `step2`, `step3` and `step4` folders, so you can compare your code with a working version at any point.

Why C, and why the Uzebox?

C is one of the most widely used programming languages in the world. Operating systems, game consoles, microcontrollers and countless programs are written in it. Learning C is worth it because:

- **It is portable.** The same C code can be compiled for almost any processor, from a tiny 8-bit microcontroller like the Uzebox's to a PC. Compilers for it exist on every platform.
- **It is efficient and fast.** C is compiled to machine code with very little overhead, which matters on a machine with 64 KB of flash memory and 4 KB of RAM.
- **It shows you how computers work.** You deal directly with bytes, memory addresses and hardware registers, so you learn what other languages hide.
- **It opens doors.** Once you know C, languages such as C++, C#, Java and JavaScript look familiar.

The **Uzebox** is an ideal place to learn it. It is a fully open source games console: the hardware design, the kernel (the system code your game is built on), the emulators and the tools are all open source and available to study. The whole system is small enough to understand, and you can even build the hardware yourself. When something goes wrong there is no hidden operating system in the way: just your code, the kernel and a microcontroller.

SPI RAM and video mode 748

The Uzebox's ATmega644 microcontroller has only **4 KB of RAM**. Most Uzebox video modes keep the screen's tile map and sprite graphics in that RAM, which leaves little for your game.

Many Uzebox boards (such as the Uzebox Omega) also carry an **SPI RAM** chip: 128 KB of extra memory, connected to the microcontroller by a fast serial link called SPI. The microcontroller can't run code from it, but it can read and write data there very quickly.

Video mode 748 is designed around SPI RAM. It reads the screen's tile maps and the sprite graphics from SPI RAM while drawing the picture, so they don't use the precious 4 KB of RAM. It also draws 192 pixels across in 16 colours, chosen from the Uzebox's 256. Mode 748 needs SPI RAM, so check that your board has it; the emulators provide it automatically.

What you need

- **The Uzebox source code**, which contains the kernel:

```
git clone https://github.com/Uzebox/uzebox.git
```

- **The AVR C compiler**: avr-gcc with avr-libc, plus `make`.

The instructions for installing avr-gcc and the rest of the software required to build Uzebox programs for several platforms including Linux, Windows, macOS, *BSD UNIX and Haiku can be found in the Software Installation section of the Getting Started on the Uzebox Uzebox wiki page.

One of the strengths of the Uzebox are its lightweight and very portable SDK and set of development tools.

- **CUzeBox-ex** (CUEX), the official and the most feature rich Uzebox emulator has ready-to-run binaries and executables for Linux, Windows and macOS available on github.

Make a folder for the game inside the Uzebox source, next to the other demos:

```
mkdir uzebox/demos/pong
cd uzebox/demos/pong
```

A short C crash course

Here is the C you need for this tutorial. Don't worry about memorising it: you'll see every part of it used in the game.

Comments are notes for people; the compiler ignores them:

```
/* This is a comment. */
```

Variables hold values. Each has a **type** saying what kind of value it holds. The Uzebox headers define short names for the common ones:

```
u8  lives = 3;        /* unsigned 8-bit: 0 to 255 */
u16 score = 1000;     /* unsigned 16-bit: 0 to 65535 */
int speed = -2;       /* signed: can be negative */
bool alive = true;    /* true or false */
```

Arithmetic and comparisons work as you'd expect. `x += 2` is short for `x = x + 2`, and `x++` adds one. Comparisons are written `==` (equal), `!=` (not equal), `<`, `>`, `<=` and `>=`; `&&` means "and", `||` means "or" and `!` means "not".

if runs code only when a condition is true:

```
if (lives == 0) {
    game_over();
} else {
    lives--;
}
```

Loops repeat code. A `for` loop counts; `while (1)` repeats forever:

```
for (i = 0; i < 4; i++) {     /* i takes the values 0, 1, 2, 3 */
    draw_brick(i);
}
```

Functions are named pieces of code. They can take values (**parameters**) and give one back (the **return value**); `void` means "nothing":

```
u8 twice(u8 n)
{
    return n * 2;
}
```

Every C program starts in the function called `main`.

Arrays hold several values of the same type, numbered from 0:

```
u8 table[3] = { 10, 20, 30 };    /* table[0] is 10, table[2] is 30 */
```

Structs group values under one name. Mode 748 uses one for each row of the screen, and you reach its parts with a dot: `rows[5].config = 0;`.

#include pulls in definitions from other files, and **#define** gives a name to a value, so the code says what a number means:

```
#include <uzebox.h>
#define PADDLE_H 32
```

Bits. Graphics often pack several values into one byte. `<<` shifts bits left, `|` combines them and `&` tests them: `(a << 4) | b` puts the 4-bit value `a` in the top half of a byte and `b` in the bottom half.

Flash and RAM. `const` data marked `PROGMEM` stays in the microcontroller's 64 KB of flash instead of being copied into its small RAM. You read it with `pgm_read_byte()`.

Addresses. `&x` means "the memory address of `x`". Mode 748 needs to be told where some things are in memory, so you'll see a few of these.

Step 1: the court

The Makefile

A **Makefile** tells `make` how to compile the kernel and your game and link them into one program. Create a file called `Makefile`:

```
# Makefile for the Uzebox Mode 748 Pong tutorial
GAME       = pong
MCU        = atmega644
# Where is the Uzebox kernel? make looks in the usual places next to or around
# this folder; if yours is elsewhere, run:  make UZEBOX=/path/to/uzebox
KERNEL_DIR ?= $(if $(UZEBOX),$(UZEBOX)/kernel,$(patsubst %/uzebox.h,%,$(firstword \
              $(wildcard ../../kernel/uzebox.h ../../../kernel/uzebox.h \
                         ../uzebox/kernel/uzebox.h ../../uzebox/kernel/uzebox.h))))
ifeq ($(wildcard $(KERNEL_DIR)/uzebox.h)$(filter clean,$(MAKECMDGOALS)),)
$(error Uzebox kernel not found. Run: make UZEBOX=/path/to/uzebox)
endif

CC      = avr-gcc
OPTIONS = -DVIDEO_MODE=748 -DINTRO_LOGO=0 -DSOUND_MIXER=MIXER_TYPE_INLINE
CFLAGS  = -mmcu=$(MCU) -Wall -std=gnu99 -Os -DF_CPU=28636360UL -fsigned-char \
          -fno-toplevel-reorder $(OPTIONS) -include m74cfg.h -I$(KERNEL_DIR) -I.
ASFLAGS = $(CFLAGS) -x assembler-with-cpp
LDFLAGS = -mmcu=$(MCU) -Wl,--section-start=.rectb=0xE000

KERNEL  = uzeboxVideoEngineCore uzeboxCore uzeboxSoundEngine \
          uzeboxSoundEngineCore uzeboxVideoEngine spiram
OBJECTS = $(addsuffix .o,$(KERNEL)) main.o

all: $(GAME).hex

%.o: $(KERNEL_DIR)/%.s
    $(CC) $(ASFLAGS) -c $< -o $@
%.o: $(KERNEL_DIR)/%.c
    $(CC) $(CFLAGS) -c $< -o $@
main.o: main.c m74cfg.h
    $(CC) $(CFLAGS) -c $< -o $@

$(GAME).elf: $(OBJECTS)
    $(CC) $(LDFLAGS) $(OBJECTS) -o $@
$(GAME).hex: $(GAME).elf
    avr-objcopy -O ihex -R .eeprom $< $@
    avr-size $<

clean:
    rm -f *.o *.elf *.hex
```

Important: the indented lines under each `name:` line must start with a **Tab**, not spaces. If `make` reports "missing separator", replace the leading spaces with a Tab.

What it does: - `KERNEL_DIR` is where the Uzebox kernel is. `make` looks for it in the usual places around your game's folder (such as `uzebox/demos/pong`, as here); if it isn't found, it tells you to run `make UZEBOX=/path/to/uzebox`, giving the folder you cloned the Uzebox source into. - `OPTIONS` selects video mode 748, switches off the Uzebox logo at start-up, and picks the sound mixer. - `CFLAGS` are the compiler settings: the ATmega644 processor, its 28.6 MHz clock, optimise for size (`-Os`), and read `m74cfg.h` first. - `LDFLAGS` places the recolour table (see below) at a fixed flash address. - The rules compile each kernel file and `main.c`, link them into `pong.elf`, then make `pong.hex`, the file you run.

The mode 748 settings

Mode 748 is configured with a few `#defines`. Create `m74cfg.h`:

```
/* Mode 748 settings for the Pong tutorial */
#ifndef M74CFG_H
#define M74CFG_H

#define M74_SPR_ENABLE   1        /* use sprites */
#define M74_MSK_ENABLE   0        /* no sprite masks */
#define M74_RECTB_OFF    0xE000   /* recolour table: the Makefile puts it here */

#define RT_BASE          64U      /* RAM tiles for sprites start at 64 * 32 = 0x0800 */
#define RT_MAX           24U      /* and there are 24 of them (up to 0x0B00) */

#endif
```

Sprites in mode 748 are drawn into **RAM tiles**: small 8 × 8 pixel tiles in normal RAM, which are placed over the background wherever a sprite is. These settings reserve 24 RAM tiles (768 bytes) starting at address 0x0800. Your program's own variables must stay below that address, which is easy for a game this size.

The program

Now the program. Create `main.c`:

```
/* Pong for the Uzebox, video mode 748 - a beginner's tutorial game */
#include <stdbool.h>
#include <stddef.h>
#include <avr/pgmspace.h>
#include <uzebox.h>
#include <spiram.h>

/* ---------------------------------------------------------------- graphics */

/* Two 4-bit pixels packed into one byte: the left pixel in the high half */
#define PX(a, b)  (((a) << 4) | (b))

/* Background tiles (8 x 8 pixels, 4 bytes per row, 32 bytes per tile).
** They stay in flash; Mode 748 needs them to start on a 256-byte boundary. */
static const u8 bg_tiles[3 * 32] __attribute__((aligned(256))) PROGMEM = {
    /* tile 0: empty */
    0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0,
    /* tile 1: a dash of the net, in colour 2 */
    PX(0,0),PX(0,2),PX(2,0),PX(0,0),  PX(0,0),PX(0,2),PX(2,0),PX(0,0),
    PX(0,0),PX(0,2),PX(2,0),PX(0,0),  PX(0,0),PX(0,2),PX(2,0),PX(0,0),
    0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0,
    /* tile 2: wall, in colour 1 */
    0,0,0,0, 0,0,0,0, 0x11,0x11,0x11,0x11, 0x11,0x11,0x11,0x11,
    0x11,0x11,0x11,0x11, 0x11,0x11,0x11,0x11, 0,0,0,0, 0,0,0,0,
};
#define T_EMPTY 0
#define T_NET   1
#define T_WALL  2

/* The sprite system recolours every pixel through this table; an identity
** table means "draw the colours as they are". The Makefile places it at
```

```
** M74_RECTB_OFF (0xE000). */
const u8 recolor[16] __attribute__((section(".rectb"))) =
    { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 };

#define SPI_VRAM     0x0000U          /* background tile map: 24 rows x 25 bytes */

/* ------------------------------------------------------------------- video */

#define ROWS 24                       /* 24 tile rows of 8 lines = 192 lines */

static m74_mode0_vram_t rows[ROWS];   /* one control block per tile row */
static u8 rowsel[] = { 0, 255 };      /* start the display at logical line 0 */

#define R(n) ((u16)(&rows[(n) % ROWS]))
static const u16 row_table[32] PROGMEM = {
    R(0), R(1), R(2), R(3), R(4), R(5), R(6), R(7), R(8), R(9), R(10), R(11),
    R(12), R(13), R(14), R(15), R(16), R(17), R(18), R(19), R(20), R(21), R(22), R(23),
    R(24), R(25), R(26), R(27), R(28), R(29), R(30), R(31)
};

static void setup_video(void)
{
    u8 r, c, line[25];
    u8* palette = (u8*)m74_paddr;

    /* colours: 0 black, 1 white, 2 grey, 3 yellow (Uzebox colours are BBGGGRRR) */
    palette[0] = 0x00; palette[1] = 0xFF; palette[2] = 0x52; palette[3] = 0x3F;

    for (r = 0; r < ROWS; r++) {
        for (c = 0; c < 25; c++) {
            if (r == 0 || r == ROWS - 1) { line[c] = T_WALL; }
            else if (c == 12)            { line[c] = T_NET; }
            else                         { line[c] = T_EMPTY; }
        }
        SpiRamWriteFrom(0, SPI_VRAM + r * 25U, line, 25);
        rows[r].config    = 0;                      /* row mode 0: tiles */
        rows[r].bg_addr   = SPI_VRAM + r * 25U;     /* its tile map in SPI RAM */
        rows[r].t0_addr_h = (u16)bg_tiles >> 8;     /* where its tiles are */
        rows[r].t1_addr_h = (u16)bg_tiles >> 8;
    }

    SetRenderingParameters(32, ROWS * 8);
    m74_rows   = (u16)rowsel;
    m74_vaddr  = (u16)row_table;
    m74_rtbase = RT_BASE;
    m74_rtmax  = RT_MAX;
    M74_VramRestore();
    m74_config |= M74_CFG_ENABLE;                   /* switch the display on */
}

int main(void)
{
    if (SpiRamInit() == 0) { while (1) { } }     /* no SPI RAM: stop here */
    setup_video();

    while (1) {
        WaitVsync(1);                            /* wait for the next frame */
    }
}
```

How it works:

- **Background tiles.** The screen is made of 8 × 8 pixel tiles. Each pixel is a colour number from 0 to 15, stored 4 bits per pixel, two pixels per byte, so a tile is 32 bytes. The `PX(a, b)` macro packs two pixels into a byte. There are three tiles: empty, a dash of the net and a piece of wall.

- **The recolour table** is required by mode 748's sprites. It can change sprite colours as they're drawn; ours leaves them unchanged.
- **Tile rows.** Mode 748 draws the screen as rows of tiles, 8 lines each. Each row has a small control block (an `m74_mode0_vram_t` struct) saying which row mode to use (0, "tiles"), where in SPI RAM its row of tile numbers is (`bg_addr`), and where its tiles are in flash (`t0_addr_h`). `row_table` lists the addresses of the blocks for the 32 possible rows; we show 24 of them (192 lines).
- **`setup_video()`** sets the palette (colour 0 black, 1 white, 2 grey, and 3 yellow for the ball later; Uzebox colours are one byte, in the form BBGGGRRR), builds each row of tile numbers and copies it into SPI RAM with `SpiRamWriteFrom()`, fills in each row's control block, tells mode 748 where everything is, and switches the display on.
- **`main()`** starts the SPI RAM (and stops if there's none), sets up the screen, then loops forever. `WaitVsync(1)` waits for the next video frame, 60 times a second.

Build and run

```
make
```

This compiles the kernel and your game. It should finish by printing the program's size. Then run it in CUzeBox-ex:

```
cuzebox pong.hex
```

You should see the court: a wall at the top and bottom and a dashed net down the middle.

Step 2: sprites

Paddles and the ball are **sprites**: graphics drawn on top of the background, which can be anywhere on the screen. Mode 748 reads sprite graphics from SPI RAM, so we make them when the program starts and copy them there.

Add this after the recolour table:

```
/* Sprite tiles are made at start-up and copied into SPI RAM */
#define SPI_VRAM     0x0000U          /* background tile map: 24 rows x 25 bytes */
#define SPI_SPRITES  0x1000U          /* sprite tiles, 32 bytes each */
#define SPR_PADDLE   0
#define SPR_BALL     1
```

Add these functions just before `setup_video()`:

```
/* Make one 8 x 8 sprite tile from a function telling the colour of each pixel */
static void make_tile(u8 number, u8 (*pixel)(u8 x, u8 y, u8 arg), u8 arg)
{
    u8 tile[32];
    u8 x, y;
    for (y = 0; y < 8; y++) {
        for (x = 0; x < 8; x += 2) {
            tile[y * 4 + x / 2] = PX(pixel(x, y, arg), pixel(x + 1, y, arg));
        }
    }
    SpiRamWriteFrom(0, SPI_SPRITES + number * 32U, tile, 32);
}

static u8 paddle_px(u8 x, u8 y, u8 arg) { return (x >= 2 && x <= 5) ? 1 : 0; }
static u8 ball_px(u8 x, u8 y, u8 arg)
{
    bool corner = (x == 1 || x == 6) && (y == 1 || y == 6);
    return (x >= 1 && x <= 6 && y >= 1 && y <= 6 && !corner) ? 3 : 0;
}
```

`make_tile()` builds a tile one pixel at a time and copies it into SPI RAM. Its second parameter is a **function** that says what colour each pixel should be: `paddle_px` draws a bar 4 pixels wide, and `ball_px` a small round yellow ball (colour 3).

In `setup_video()`, add these two lines just before `SetRenderingParameters`:

```
make_tile(SPR_PADDLE, paddle_px, 0);
make_tile(SPR_BALL, ball_px, 0);
```

Add this function after `setup_video()`:

```
/* Draw a sprite tile with its top left corner at (x, y) */
static void sprite(u8 number, u8 x, u8 y)
{
    M74_BlitSprite(SPI_SPRITES + number * 32U, x + 8, y + 8, 0);
}
```

Finally, replace `main()` with:

```
/* ------------------------------------------------------------------- game */

#define TOP      8                  /* the court, inside the walls */
#define BOTTOM   184
#define PADDLE_H 32                 /* 4 sprite tiles tall */
#define LEFT_X   8                  /* sprite x of each paddle */
#define RIGHT_X  176

u8 left_y = 80, right_y = 80;       /* paddle tops */
int ball_x = 92, ball_y = 92;       /* ball position */

static void draw_paddle(u8 x, u8 y)
{
    u8 i;
    for (i = 0; i < 4; i++) { sprite(SPR_PADDLE, x, y + i * 8); }
}

static void draw(void)
{
    M74_VramRestore();                          /* remove last frame's sprites */
    draw_paddle(LEFT_X, left_y);
    draw_paddle(RIGHT_X, right_y);
    sprite(SPR_BALL, ball_x, ball_y);
}

int main(void)
{
    if (SpiRamInit() == 0) { while (1) { } }    /* no SPI RAM: stop here */
    setup_video();

    while (1) {
        WaitVsync(1);                           /* wait for the next frame */
        draw();
    }
}
```

Mode 748 draws sprites into RAM tiles, and they stay until removed. So each frame `draw()` first calls `M74_VramRestore()` to clear the old sprites, then draws everything again. A paddle is 32 pixels tall, so it is drawn as 4 sprite tiles stacked up. `M74_BlitSprite()` wants the position of the sprite's bottom right corner plus one, which is why `sprite()` adds 8.

Build and run: the two paddles and the ball appear, standing still.

Step 3: movement

Now the game comes alive. Replace everything from the `game` comment line to the end of the file with:

```
/* ---------------------------------------------------------------- game */

#define TOP      8                  /* the court, inside the walls */
#define BOTTOM   184
#define PADDLE_H 32                 /* 4 sprite tiles tall */
#define LEFT_X   8                  /* sprite x of each paddle */
#define RIGHT_X  176

u8 left_y = 80, right_y = 80;       /* paddle tops */
int ball_x = 92, ball_y = 92;       /* ball position */
int speed_x = 2, speed_y = 1;       /* ball speed in pixels per frame */

static void draw_paddle(u8 x, u8 y)
{
    u8 i;
    for (i = 0; i < 4; i++) { sprite(SPR_PADDLE, x, y + i * 8); }
}

static void serve(int direction)
{
    ball_x = 92; ball_y = 92;
    speed_x = direction * 2;
    speed_y = (speed_y > 0) ? -1 : 1;
}

/* Does the ball touch a paddle whose sprite is at (x, y)? */
static bool hits(u8 x, u8 y)
{
    return ball_x + 1 <= x + 5 && ball_x + 6 >= x + 2 &&
           ball_y + 6 >= y && ball_y + 1 <= y + PADDLE_H - 1;
}

static void bounce(u8 paddle_y)
{
    int offset = (ball_y + 4) - (paddle_y + PADDLE_H / 2);   /* -16 .. 16 */
    speed_x = -speed_x;
    speed_y = offset / 6;                  /* hitting off-centre angles the ball */
}

static void update(void)
{
    u16 pad = ReadJoypad(0);

    /* player: up and down */
    if ((pad & BTN_UP) && left_y > TOP) { left_y -= 2; }
    if ((pad & BTN_DOWN) && left_y + PADDLE_H < BOTTOM) { left_y += 2; }

    /* computer: follows the ball, a little slower than it can move */
    if (ball_y + 4 < right_y + PADDLE_H / 2 && right_y > TOP) { right_y--; }
    if (ball_y + 4 > right_y + PADDLE_H / 2 && right_y + PADDLE_H < BOTTOM) { right_y++; }

    /* ball */
    ball_x += speed_x;
    ball_y += speed_y;
    if (ball_y + 1 < TOP)     { ball_y = TOP - 1;    speed_y = -speed_y; }
    if (ball_y + 6 >= BOTTOM) { ball_y = BOTTOM - 7; speed_y = -speed_y; }
    if (speed_x < 0 && hits(LEFT_X, left_y))   { bounce(left_y); }
    if (speed_x > 0 && hits(RIGHT_X, right_y)) { bounce(right_y); }

    /* missed: serve again */
    if (ball_x < 0)   { serve(1); }
    if (ball_x > 186) { serve(-1); }
```

```
}

static void draw(void)
{
    M74_VramRestore();                          /* remove last frame's sprites */
    draw_paddle(LEFT_X, left_y);
    draw_paddle(RIGHT_X, right_y);
    sprite(SPR_BALL, ball_x, ball_y);
}

int main(void)
{
    if (SpiRamInit() == 0) { while (1) { } }    /* no SPI RAM: stop here */
    setup_video();

    while (1) {
        WaitVsync(1);                           /* wait for the next frame */
        draw();
        update();
    }
}
```

How it works: - **The player.** `ReadJoypad(0)` returns the buttons held on controller 1, one bit each. `pad & BTN_UP` is true when Up is held. The checks against `TOP` and `BOTTOM` keep the paddle inside the court. - **The computer** moves its paddle 1 pixel per frame towards the ball, slower than the player's 2, so it can be beaten. - **The ball** moves by `speed_x` and `speed_y` pixels every frame. When it reaches a wall, its vertical speed changes sign, so it bounces. - **Collisions.** `hits()` checks whether the ball's square overlaps a paddle. `bounce()` sends it back, and the further from the paddle's centre it hits, the steeper the angle, which gives the player some control. - **Missed balls** are served again from the middle by `serve()`.

Build and run, and play with the arrow keys (controller 1's Up and Down).

Step 4: scores and sound

To finish the game, add scores and a sound effect.

Score digits. Add the last sprite number and a small font after `#define SPR_BALL 1`:

```
#define SPR_DIGIT0   2                /* digits 0-9 follow */

/* Score digits, 4 x 7 pixels: one byte per row, the low 4 bits are the pixels */
static const u8 font[10][7] PROGMEM = {
    {6,9,9,9,9,9,6}, {2,6,2,2,2,2,7}, {6,9,1,2,4,8,15}, {14,1,1,6,1,1,14},
    {2,6,10,15,2,2,2}, {15,8,14,1,1,9,6}, {6,8,8,14,9,9,6}, {15,1,2,2,4,4,4},
    {6,9,9,6,9,9,6}, {6,9,9,7,1,1,6}
};
```

Each digit is 4 × 7 pixels, one byte per row; a bit set to 1 means "draw this pixel". Add the function that turns a digit into a sprite tile, after `ball_px`:

```
static u8 digit_px(u8 x, u8 y, u8 d)
{
    if (y >= 7) { return 0; }
    return (pgm_read_byte(&font[d][y]) & (8 >> (x / 2))) ? 1 : 0;
}
```

`8 >> (x / 2)` picks the bit for column `x`; each bit is drawn 2 pixels wide, so the digits fill the 8-pixel tile. In `setup_video()`, after the two `make_tile` lines, add:

```
for (c = 0; c < 10; c++) { make_tile(SPR_DIGIT0 + c, digit_px, c); }
```

A sound effect. Uzebox sounds are described by **patches**: short lists of commands, each with a delay (in frames), a command and a value. Add this before the `game` comment line:

```
/* ------------------------------------------------------------------- sound */

static const char bounce_sound[] PROGMEM = {
    0, PC_WAVE, 4,            /* a square wave */
    0, PC_PITCH, 72,          /* a high note */
    0, PC_ENV_SPEED, -40,     /* that fades out quickly */
    4, PC_NOTE_CUT, 0,        /* and stops after 4 frames */
    0, PATCH_END
};
static const struct PatchStruct patches[] PROGMEM = { {0, NULL, bounce_sound, 0, 0} };
```

See here for more details on the Uzebox sound engine.

UzeSynth makes it easier to edit and hear Uzebox patches.

The finished game section. Replace everything from the `game` comment line to the end with:

```
/* ------------------------------------------------------------------- game */

#define TOP      8                  /* the court, inside the walls */
#define BOTTOM   184
#define PADDLE_H 32                 /* 4 sprite tiles tall */
#define LEFT_X   8                  /* sprite x of each paddle */
#define RIGHT_X  176

u8 left_y = 80, right_y = 80;       /* paddle tops */
int ball_x = 92, ball_y = 92;       /* ball position */
int speed_x = 2, speed_y = 1;       /* ball speed in pixels per frame */
u8 score_left = 0, score_right = 0;

static void draw_paddle(u8 x, u8 y)
{
    u8 i;
    for (i = 0; i < 4; i++) { sprite(SPR_PADDLE, x, y + i * 8); }
}

static void serve(int direction)
{
    ball_x = 92; ball_y = 92;
    speed_x = direction * 2;
    speed_y = (speed_y > 0) ? -1 : 1;
}

/* Does the ball touch a paddle whose sprite is at (x, y)? */
static bool hits(u8 x, u8 y)
{
    return ball_x + 1 <= x + 5 && ball_x + 6 >= x + 2 &&
           ball_y + 6 >= y && ball_y + 1 <= y + PADDLE_H - 1;
}

static void bounce(u8 paddle_y)
{
    int offset = (ball_y + 4) - (paddle_y + PADDLE_H / 2);   /* -16 .. 16 */
    speed_x = -speed_x;
    speed_y = offset / 6;                  /* hitting off-centre angles the ball */
    TriggerFx(0, 0xC0, true);
}

static void update(void)
{
    u16 pad = ReadJoypad(0);

    /* player: up and down */
    if ((pad & BTN_UP) && left_y > TOP) { left_y -= 2; }
    if ((pad & BTN_DOWN) && left_y + PADDLE_H < BOTTOM) { left_y += 2; }
```

```
    /* computer: follows the ball, a little slower than it can move */
    if (ball_y + 4 < right_y + PADDLE_H / 2 && right_y > TOP) { right_y--; }
    if (ball_y + 4 > right_y + PADDLE_H / 2 && right_y + PADDLE_H < BOTTOM) { right_y++; }

    /* ball */
    ball_x += speed_x;
    ball_y += speed_y;
    if (ball_y + 1 < TOP)     { ball_y = TOP - 1;    speed_y = -speed_y; TriggerFx(0,
        0x80, true); }
    if (ball_y + 6 >= BOTTOM) { ball_y = BOTTOM - 7; speed_y = -speed_y; TriggerFx(0,
        0x80, true); }
    if (speed_x < 0 && hits(LEFT_X, left_y))   { bounce(left_y); }
    if (speed_x > 0 && hits(RIGHT_X, right_y)) { bounce(right_y); }

    /* points */
    if (ball_x < 0)   { score_right++; serve(1); }
    if (ball_x > 186) { score_left++;  serve(-1); }
    if (score_left > 9 || score_right > 9) { score_left = 0; score_right = 0; }
}

static void draw(void)
{
    M74_VramRestore();                          /* remove last frame's sprites */
    sprite(SPR_DIGIT0 + score_left, 72, 12);
    sprite(SPR_DIGIT0 + score_right, 112, 12);
    draw_paddle(LEFT_X, left_y);
    draw_paddle(RIGHT_X, right_y);
    sprite(SPR_BALL, ball_x, ball_y);
}

int main(void)
{
    if (SpiRamInit() == 0) { while (1) { } }    /* no SPI RAM: stop here */
    InitMusicPlayer(patches);
    setup_video();

    while (1) {
        WaitVsync(1);                           /* wait for the next frame */
        draw();
        update();
    }
}
```

What's new: - `score_left` and `score_right` count the points. A missed ball gives a point to the other player, and the scores go back to 0 after 9. - `draw()` draws each score as a digit sprite: sprite number `SPR_DIGIT0 + score` is the right digit. - `InitMusicPlayer(patches)` tells the sound engine about our patch, and `TriggerFx(0, volume, true)` plays it: louder off a paddle, quieter off a wall.

Build and run: you have a complete game of Pong.

Running it on a Uzebox

To play on real hardware, the Uzebox's SD card loader needs a `.uze` file. Build the `packrom` tool from `uzebox/tools/packrom`, then:

```
packrom pong.hex pong.uze gameinfo.properties
```

where `gameinfo.properties` is a small text file with your game's details (see the other demos for examples). Copy `pong.uze` to the SD card. Remember that mode 748 needs a board with SPI RAM.

Where next?

Some ideas for further practice:

- Let a second player control the right paddle with `ReadJoypad(1)`.
- Make the ball speed up after each hit, or end the game at 5 points with a winner shown.
- Draw your own tiles and sprites, or add more colours and sounds.

Further reading

- Check out the Uzebox API.
- Read the mode 748 manual to learn about its other row modes, scrolling and more.
- Peruse the code for IK 748 and try to understand how it works.
- Scope the Uzebox wiki's C Tips, Tricks and Optimizations page.

Uzebox developers requiring more low-level AVR information should refer to the avr-libc manual pages.

If you have Uzebox related questions you may get an answer on the Uzebox forum.

The complete program

The finished program is `step4/main.c`.